

## DAY 6 TASKS

Nandini Bhimineni

Yoganandini9@gmail.com

### Task 1 — User Authentication System

Create a complete authentication module using Node.js, Express.js, MongoDB, bcrypt, and JWT.

#### Requirements

- Create Register API
- Create Login API
- Hash passwords using bcrypt
- Generate JWT token after successful login
- Store users in MongoDB

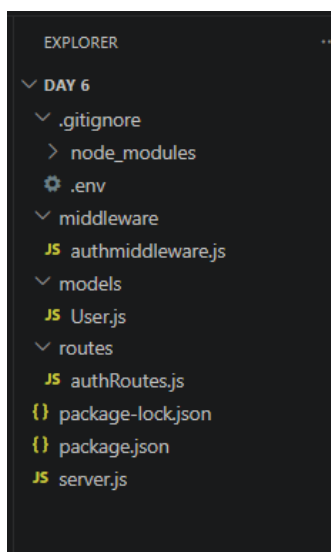
#### APIs Required

POST /auth/register

POST /auth/login

#### Expected Output

- User data stored securely
- Passwords encrypted
- JWT token generated successfully



```

{} package.json > ...
1  {
    >Debug
2    "scripts":{
3      "start":"nodemon server.js"
4    },
5    "dependencies": {
6      "bcryptjs": "^3.0.3",
7      "cors": "^2.8.6",
8      "dotenv": "^17.4.2",
9      "express": "^5.2.1",
10     "jsonwebtoken": "^9.0.3",
11     "mongoose": "^9.6.2"
12   },
13   "devDependencies": {
14     "nodemon": "^3.1.14"
15   }
16 }
17

```

```

Welcome JS server.js JS authmiddleware.js X .env JS User.js JS authRoutes.js
middleware > JS authmiddleware.js > ...
1  const jwt = require("jsonwebtoken");
2
3  const authMiddleware = (req, res, next) => {
4
5    try {
6
7      const token = req.header("Authorization");
8
9      if (!token) {
10       return res.status(401).json({
11         message: "Access Denied"
12       });
13     }
14
15     const verified = jwt.verify(token, process.env.JWT_SECRET);
16
17     req.user = verified;
18
19     next();
20   } catch (error) {
21
22     res.status(401).json({
23       message: "Invalid Token"
24     });
25   }
26 }
27 };
28
29 module.exports = authMiddleware;

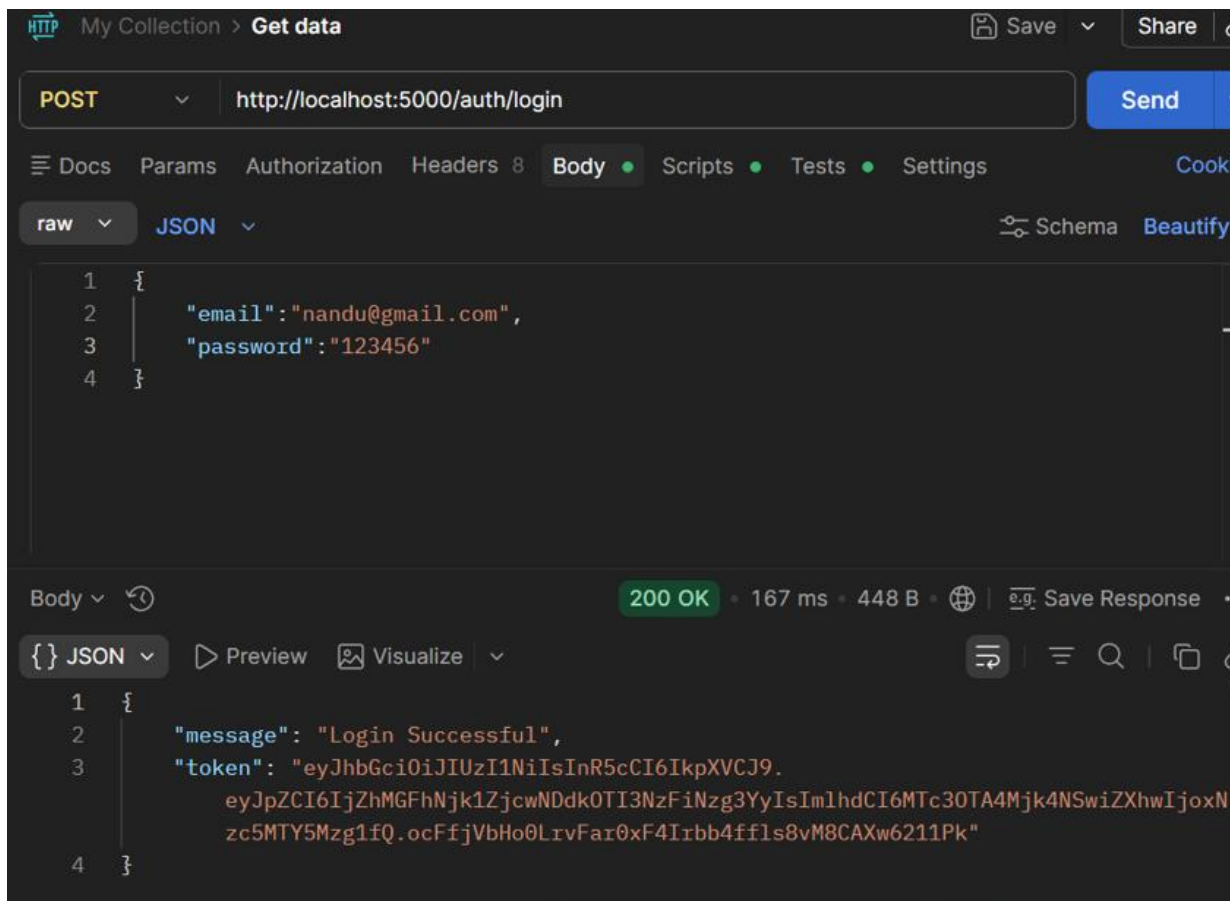
```

```
models > JS User.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4
5    name: {
6      type: String,
7      required: true
8    },
9
10   email: {
11     type: String,
12     required: true,
13     unique: true
14   },
15
16   password: {
17     type: String,
18     required: true
19   }
20 },
21 });
22
23 module.exports = mongoose.model("User", userSchema);
```

```
JS server.js > app.listen() callback
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const dotenv = require("dotenv");
4
5  const authRoutes = require("../routes/authRoutes");
6
7  dotenv.config();
8
9  const app = express();
10
11  app.use(express.json());
12
13  mongoose.connect(process.env.MONGO_URI)
14    .then(() => console.log("MongoDB Connected"))
15    .catch((err) => console.log(err));
16
17  app.use("/auth", authRoutes);
18
19  app.listen(5000, () => {
20    console.log("Server running on port 5000");
21  });
```

```
PS C:\Users\yogan\Downloads\Raritone\DAY 6> npm start
> start
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
Server running on port 5000
MongoDB Connected
```



### JWT Token is generated Successfully

## **Task 2 — Protected Route Implementation**

**Create protected APIs using JWT middleware.**

### **Requirements**

- **Create authentication middleware**
- **Verify JWT token**
- **Restrict unauthorized access**
- **Return proper error messages for invalid tokens**

### **Protected API Example**

**GET /profile**

### **Expected Output**

- **Authorized users can access route**
- **Unauthorized users receive access denied message**

WE

- Created authMiddleware.js file inside middleware folder
- Used jwt.verify() to verify JWT token
- Read token from Authorization header
- Protected the /auth/profile route using middleware
- Restricted unauthorized users from accessing the route
- Returned proper error messages like “Access Denied” and “Invalid Token”

Files Used:

1. middleware/authMiddleware.js
2. routes/authRoutes.js

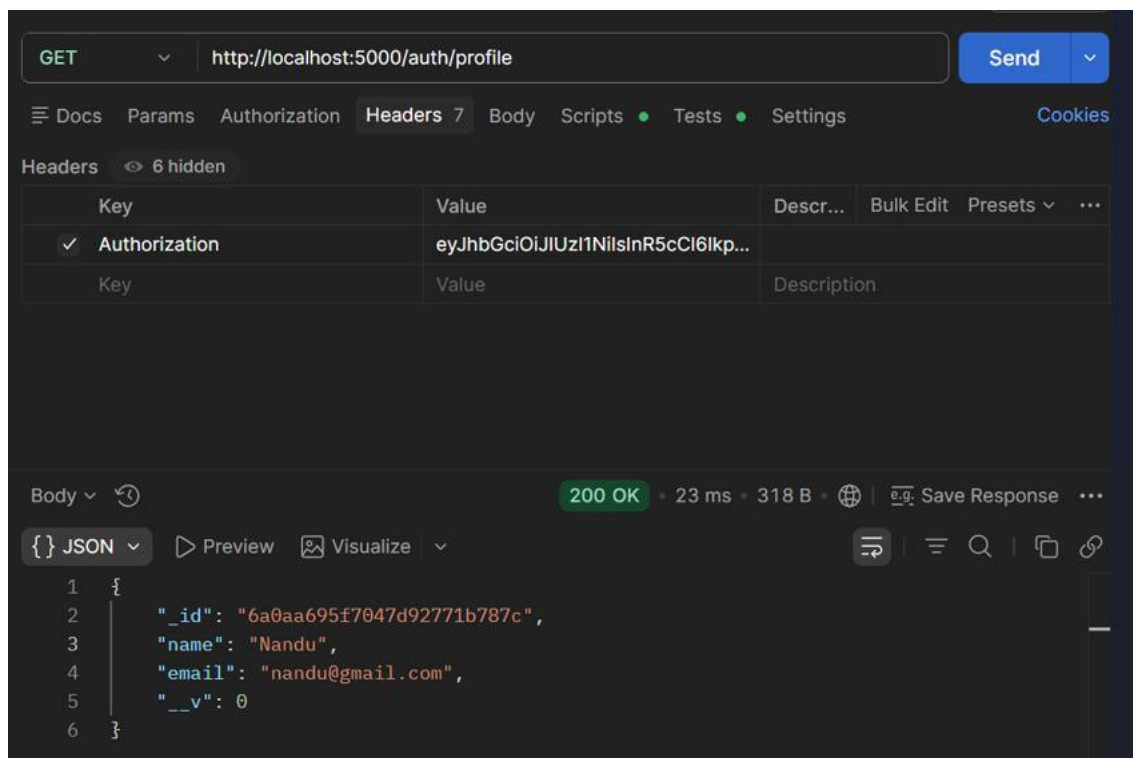
Protected Route Used:

GET /auth/profile

Middleware Implementation:

- Checked whether token exists
- Verified token using JWT secret key
- Allowed valid users to continue using next()

- Blocked invalid users with error response



### Task 3 — CRUD API Enhancement

Improve existing CRUD APIs with proper validations and structured responses.

#### Requirements

- Validate required fields
- Prevent empty submissions
- Return proper status codes
- Handle database errors properly

#### APIs to Test

POST

GET

PUT

DELETE

```

// EMPTY FIELD VALIDATION
if (!name || !email || !password) {
  return res.status(400).json({
    message: "All fields are required"
  });
}

// EMAIL VALIDATION
if (!email.includes("@")) {
  return res.status(400).json({
    message: "Invalid Email Format"
  });
}

// PASSWORD VALIDATION
if (password.length < 6) {
  return res.status(400).json({
    message: "Password must be at least 6 characters"
  });
}
```

## Task 4 — API Testing using Postman

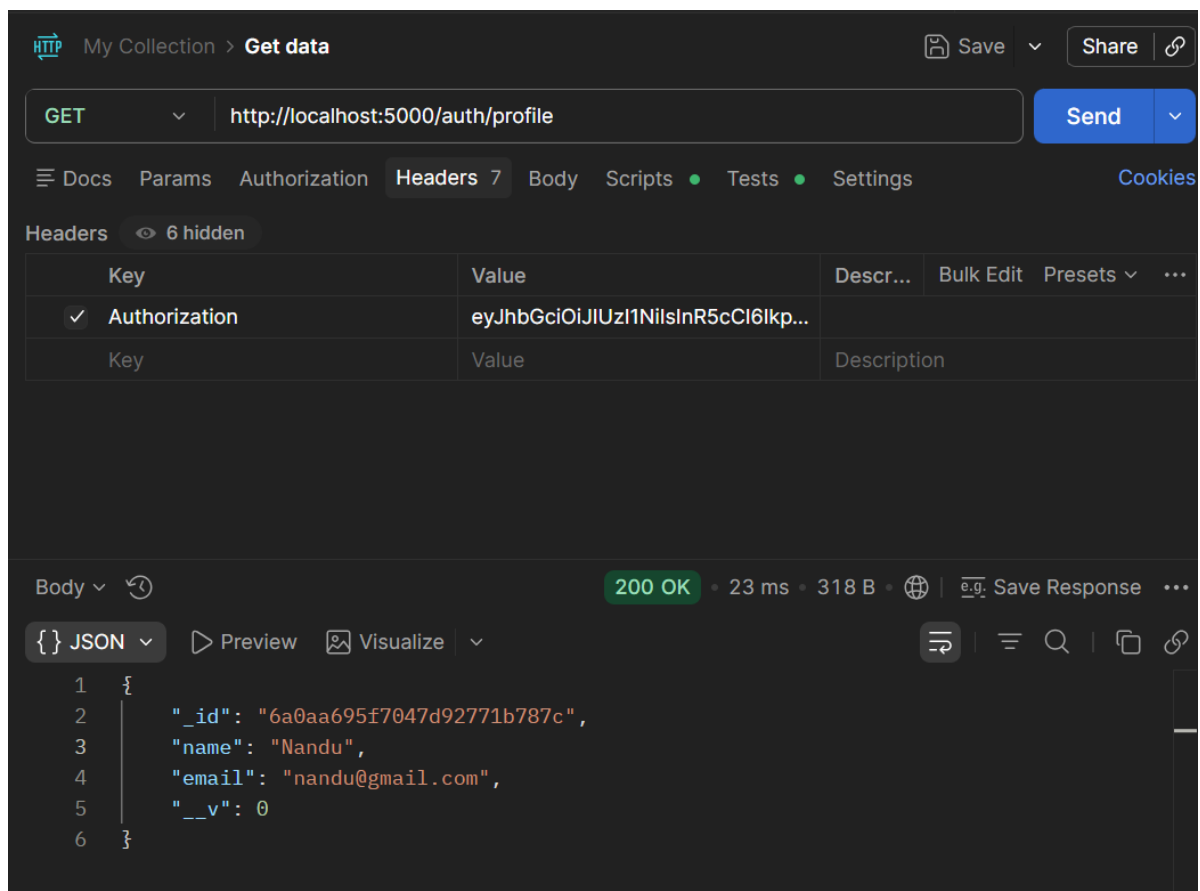
Test all backend APIs using Postman.

### Requirements

- Test Register API
- Test Login API
- Test Protected Route
- Test CRUD APIs

### Deliverables

- Screenshots of API responses
- JWT token testing
- Postman collection export (optional)



HTTP My Collection > Get data Save Share

**POST** ▼  Send ▼

Docs Params Authorization Headers 8 **Body** Scripts Tests Settings Cookies

**raw** ▼ **JSON** ▼ Schema Beautify

```
1 {
2   "email": "nandu@gmail.com",
3   "password": "123456"
4 }
```

Body ▼ 🔄 200 OK • 167 ms • 448 B • 🌐 | e.g. Save Response ⋮

**{}** **JSON** ▼ ▶ Preview 🖼 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "message": "Login Successful",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGFhNjk1ZjcwNDdkOTI3NzFiNzg3YyIsImhhdCI6MTc3OTA4Mjk4NSwiZXhwIjoxNzc5MTY5Mzg1fQ.ocFfjVbHo0LrvFar0xF4Irb4ffls8vM8CAXw6211Pk"
4 }
```

**POST** ▼  Send ▼

Docs Params Authorization Headers 8 **Body** Scripts Tests Settings Cookies

**raw** ▼ **JSON** ▼ Schema Beautify

```
1 {
2   "name": "Bob",
3   "email": "bob@gmail.com",
4   "password": "bob123"
5 }
```

Body ▼ 🔄 201 Created • 181 ms • 282 B • 🌐 | e.g. Save Response ⋮

**{}** **JSON** ▼ ▶ Preview 🖼 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "message": "User Registered Successfully"
3 }
```

POST http://localhost:5000/auth/register

Docs Params Authorization Headers 8 Body Scripts Tests Settings

raw JSON Schema

```
1 {
2   "name": "",
3   "email": "",
4   "password": ""
5 }
```

Body 400 Bad Request • 38 ms • 281 B • Save Response

JSON Preview Debug with AI

```
1 {
2   "message": "All fields are required"
3 }
```

My Collection > Get data Save Share

POST http://localhost:5000/auth/register Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

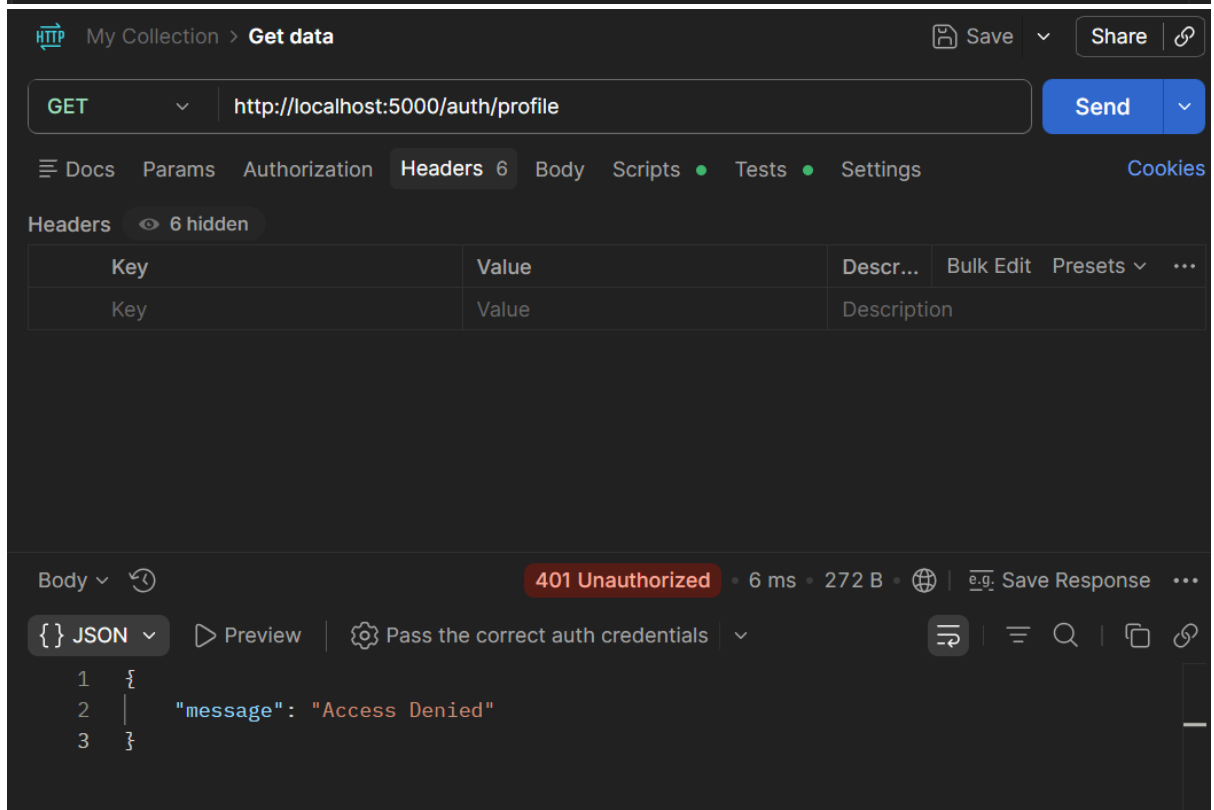
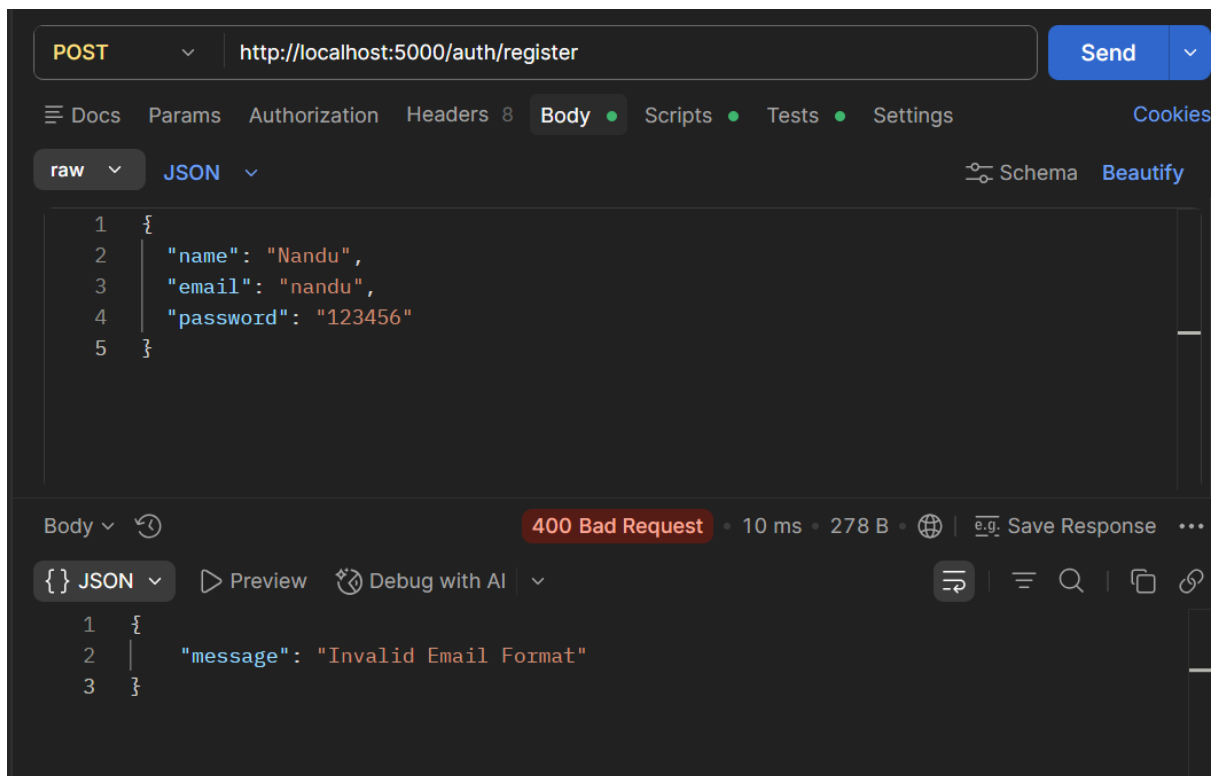
raw JSON Schema Beautify

```
1 {
2   "name": "Nandu",
3   "email": "nandu@gmail.com",
4   "password": "123"
5 }
```

Body 400 Bad Request • 4 ms • 296 B • Save Response

JSON Preview Debug with AI

```
1 {
2   "message": "Password must be at least 6 characters"
3 }
```



## Task 1 — User Authentication System

User Authentication System is used to securely verify users before allowing access to an application. In this project, Node.js and Express.js were used to create APIs for user registration and login. MongoDB was used to store user information. Passwords were encrypted using bcrypt before storing them in the database to improve security. After

successful login, JWT (JSON Web Token) was generated to authenticate users in future requests.

#### Task 2 — Protected Route Implementation

Protected routes are used to restrict unauthorized users from accessing sensitive APIs. JWT middleware was created to verify tokens before allowing access to protected routes. The middleware checks whether the token exists and validates it using `jwt.verify()`. If the token is valid, the user is allowed to access the route; otherwise, an error message is returned.

#### Task 3 — CRUD API Enhancement

CRUD operations include Create, Read, Update, and Delete functionalities. Validations were added to ensure that users provide correct data before storing it in the database. Empty fields, invalid email formats, and short passwords were handled properly. Status codes and try-catch blocks were used to improve error handling and API responses.

#### Task 4 — API Testing using Postman

Postman is an API testing tool used to test backend APIs. All APIs such as Register, Login, Protected Route, Update, and Delete were tested using Postman. Request methods, headers, request bodies, and responses were verified to ensure that APIs worked correctly.